

Approach Document - Conversational SHL Assessment Recommender

Design summary

I built a stateless FastAPI service for recommending SHL Individual Test Solutions. The API has the two required endpoints, 'GET /health' and 'POST /chat'. Each chat request carries the full conversation history, so the service reconstructs the user intent from scratch on every call and stores no per-conversation state. The agent has five routes: clarify, recommend, refine, compare and refuse. Recommendations are always built from the local SHL catalog JSON and every returned URL is validated against that catalog before the response is sent.

The main design decision was to keep the agent deterministic around safety-critical behavior. The language-model-like part of the system is not allowed to invent product names or URLs. The controller decides whether enough context exists, whether the request is off-topic, whether the latest user turn is a refinement and whether the user is asking for a comparison. This reduces failure on behavior probes and protects the hard evaluation requirements.

Catalog and retrieval setup

The scraper collects SHL product pages from the product catalog restricted to Individual Test Solutions. It stores each assessment with name, canonical URL, description, job levels, languages, duration, test type, aliases and source. The runtime catalog loader rejects invalid records and only accepts URLs under '/solutions/products/product-catalog/view'.

Retrieval uses a hybrid method instead of pure embeddings. Each assessment is indexed using BM25-style lexical retrieval and TF-IDF similarity over a rich search string made from the product name, description, job levels, languages, test type label and aliases. A metadata scorer then boosts exact skill matches, requested test types and role-specific fits. This matters because many SHL products are exact-skill assessments such as Java, SQL, Excel or Numerical Reasoning, where lexical matching is often stronger than semantic search. For broad or ambiguous valid requests, the system returns up to 10 recommendations to optimize Recall@10.

Prompting and context engineering

The agent reconstructs a canonical hiring state from all user turns. The state includes role, seniority, hard skills, soft skills, requested test types, excluded test types, job description text, comparison targets, vague-query flags and refusal flags. Later user turns override or extend earlier context. For example, if the user first asks for Java assessments and then says "Actually add personality tests", the state keeps the Java role and adds test type 'P' instead of starting over.

Clarification is intentionally compact because the evaluator has an 8-turn cap. If the user says only "I need an assessment", the agent asks for role and assessment focus. If the user gives a full job description or enough role plus skill context, it recommends immediately. If the user says "no preference" to a clarification, it stops asking and recommends from available context.

Comparison is handled by fuzzy matching both requested names against catalog names and aliases, then generating the answer from local catalog fields only. Refusal happens before retrieval for prompt injection, general hiring advice, legal questions and requests for non-SHL recommendations.

Evaluation approach

I tested three groups of behavior. First, hard checks: exact '/health' response, strict response schema, 1 to 10 recommendations only when committed, empty recommendations while clarifying or refusing and catalog-only URLs. Second, behavior probes: vague query clarification, off-topic refusal, prompt-injection refusal, direct recommendation for a job description, refinement after a mid-conversation edit and grounded comparison. Third, Recall@10 on public traces when available, using the fraction of expected assessments appearing in the top 10 returned names.

What did not work well was relying only on semantic similarity or asking many clarifying questions. Pure semantic scoring ranked generic ability/personality tests above exact technical tests for developer roles. Asking many questions also wasted the 8-turn budget. The final version improved by combining lexical retrieval, TF-IDF, metadata boosts and one-question clarification. AI assistance was used to draft parts of the code scaffold and test ideas, but the retrieval logic, schema constraints, validation path and agent behavior were reviewed and kept explicit so they can be defended in a technical deep-dive.